

四階段管F中央處理器設計與實作

2026 年微處理器系統期末專案報告

實作組：楊承F (學號：113590051, 貢獻度 65%)

報告組：F楷程 (學號：113590017, 貢獻度 35%)

2026 年 6 月 26 日

實驗主題	四階段管F中央處理器設計與實作 (4-Stage Pipeline CPU Design & Implementation)
課程名稱	微處理器系統 (Microprocessor Systems, 2026)
班級	資訊工程學系
組F	第 19 組
指導老師	課程授課教授

1 實驗目的與專案要求

本期末專案的目標是在 Altera DE2-115 開發板（搭載 Cyclone IV E FPGA 晶片）上，使用 VHDL 語言設計F實作一個高效能的四階段管F中央處理器（4-Stage Pipeline CPU）。本專案的核心硬體與軟體要求如下：

- 管F化設計：所有指令的執行均須以管F（Pipeline）方式實現，管F深度F 4 階段：
 - IF (Instruction Fetch)：F取指撥開關傳入的 16 位元指令編碼。
 - ID (Instruction Decode / Register Read)：解碼指令，自 4×8-bit 暫存器檔案讀取運算元，F執行衝突判定。
 - EXE (Execution)：ALU 運算或多時序除法器執行階段。
 - WB (Write Back)：將運算結果同步寫回指定的目標暫存器。
- 資料衝突與前饋機制 (Data Hazard & Forwarding)：F消除管F中因指令資料相依性所造成的延遲，必須設計硬體資料前饋單元 (Forwarding Unit)，即時偵測F處理 EXE 階段與 WB 階段的資料衝突。使單F期指令（如 LOAD, MOVE, ADD 等）在發生衝突時能以 Zero-Stall（零停頓）連續推進，F以F色 LED 燈 LEDG[0] 指示衝突的發生。
- 多時序除法器 (Multi-cycle DIV) 與管F暫停 (Stall)：進階功能要求整合前期實驗實作的移位相F式除法器 (Restoring Division)，進行 8 位元多時序（8 個時FF期）除法運算。除法執行期間，管F控制單元必須正確暫停 (Stall) 上游階段，F在 EXE/WB 暫存器中插入氣泡 (Bubble) 以防止寫回衝突，同時以F色 LED 燈 LEDG[1] 顯示忙碌狀態 (exe_busy)。另外需包含除以零 Bypass 防錯機制：偵測到除數F 0 時跳過 8 F期 Stall，直接輸出錯誤碼 0xFF。

4. 硬體周邊控制與顯示：

- 使用兩組雙位數七段顯示器顯示 Rs (HEX1/0)、Rt (HEX3/2) 暫存器的十六進制數值 (範圍 00 ~ FF)。
- 使用三顆七段顯示器顯示當前指撥開關輸入指令 (HEX5)、IF/ID 暫存器 (HEX4)、以及 ID/EXE 暫存器 (HEX6) 的 Opcode。
- 使用紅色 LED 燈 LEDR[7:0] 即時顯示當前指撥開關輸入的 8 位元立即值 (Data)。

2 核心硬體架構與規格設計

2.1 微架構管F暫存器定義

實現四階段管F，我們在各階段邊界定義了管F暫存器，以時F正緣（即手動按鍵 KEY[0] 按下再釋放時的上升緣）進行同步更新：

- **IF/ID 暫存器：**鎖存自指撥開關輸入的指令。
 - IFID_opcode (4-bit)：存儲當前指令操作碼。
 - IFID_rs_id (2-bit)：存儲目標暫存器選擇子。
 - IFID_rt_id (2-bit)：存儲來源暫存器選擇子。
 - IFID_data (8-bit)：存儲 8 位元立即值。
- **ID/EXE 暫存器：**鎖存解碼完成F前饋後的運算元。
 - IDEXE_opcode (4-bit)
 - IDEXE_rs_id (2-bit)
 - IDEXE_rs_val (8-bit)：經由 Forwarding Mux 選擇後的 Rs 數值。
 - IDEXE_rt_id (2-bit)
 - IDEXE_rt_val (8-bit)：經由 Forwarding Mux 選擇後的 Rt 數值。
 - IDEXE_data (8-bit)：供 LOAD 指令寫回的立即值。
- **EXE/WB 暫存器：**鎖存 EXE 階段 ALU 或除法狀態機的最終結果，準備在 WB 階段寫回。
 - EXEWB_opcode (4-bit)
 - EXEWB_rs_id (2-bit)：目標暫存器 ID。
 - EXEWB_result (8-bit)：寫回的資料。

2.2 資料前饋單元 (Forwarding Unit) 設計邏輯

當後續指令讀取前序指令尚未寫入暫存器檔案的最新結果時，會產生資料 Hazard。本處理器設計了完整的前饋機制，其邏輯架構如圖 1 所示：

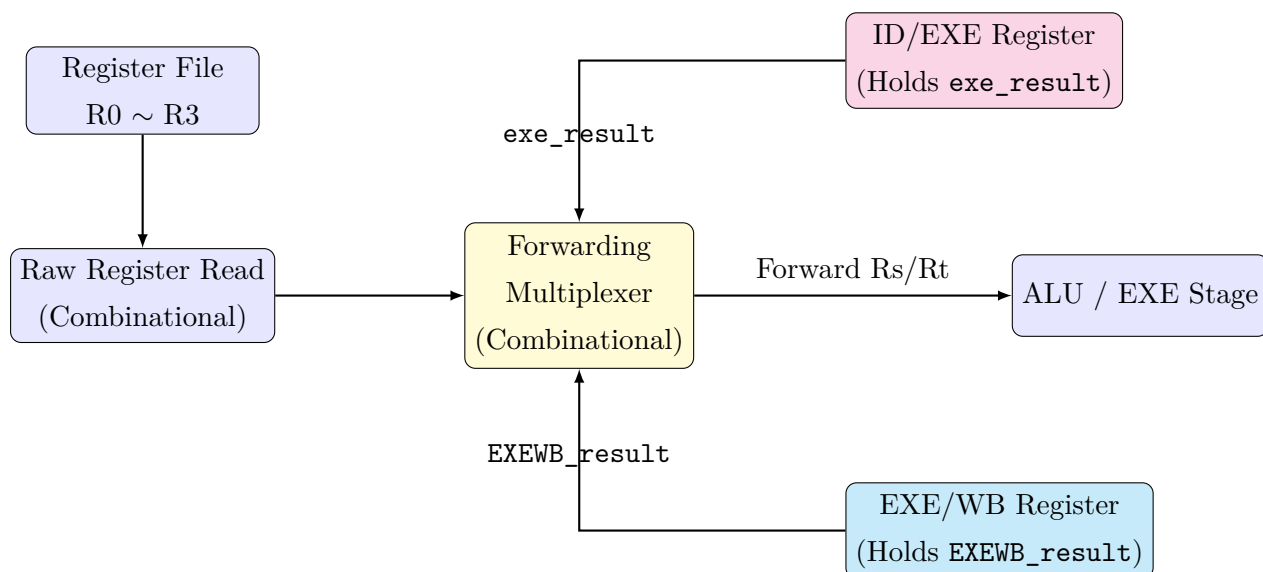


圖 1: 前饋單元 (Forwarding Unit) 邏輯架構圖

- **衝突偵測邏輯：** 首先判斷處於 ID 階段的指令是否確實需要讀取 Rs 或 Rt。例如 LOAD 僅寫入而不讀取暫存器，NOP 不讀寫暫存器。

- reads_rs 成立條件：指令 ∈ ADD, AND, SLT, SUB(A-B), NOR, SUB(B-A), DIV。

- reads_rt 成立條件：指令 ∈ 上述運算指令，或 MOVE。

- **EXE 階段衝突前饋 (EXE Forwarding)：** 若 ID 階段讀取的暫存器與處於 EXE 階段的指令目標暫存器一致，且 EXE 階段指令有效，則直接前饋 exe_result。

$$\text{fwd_rs_exe} = \text{reads_rs} \wedge \text{IDEXE_valid} \wedge (\text{IDEXE_opcode} \neq \text{NOP}) \wedge (\text{IDEXE_rs_id} = \text{IFID_rs_id}) \quad (1)$$

$$\text{fwd_rt_exe} = \text{reads_rt} \wedge \text{IDEXE_valid} \wedge (\text{IDEXE_opcode} \neq \text{NOP}) \wedge (\text{IDEXE_rs_id} = \text{IFID_rt_id}) \quad (2)$$

- **WB 階段衝突前饋 (WB Forwarding)：** 若 ID 階段讀取的暫存器與處於 WB 階段的指令目標暫存器一致，且 WB 階段指令有效，且未被 EXE 階段的最新結果覆蓋，則前饋 EXEWB_result。

$$\text{fwd_rs_wb} = \text{reads_rs} \wedge \text{EXEWB_valid} \wedge (\text{EXEWB_opcode} \neq \text{NOP}) \wedge (\text{EXEWB_rs_id} = \text{IFID_rs_id}) \quad (3)$$

$$\text{fwd_rt_wb} = \text{reads_rt} \wedge \text{EXEWB_valid} \wedge (\text{EXEWB_opcode} \neq \text{NOP}) \wedge (\text{EXEWB_rs_id} = \text{IFID_rt_id}) \quad (4)$$

- **衝突指示燈**：若上述任一前饋信號 $\overline{F}$ 真，表示目前管 $\overline{F}$ 中正發生衝突且前饋單元正即時修正資料，此時 $\overline{F}$ 色 LED 燈 LEDG[0] 亮起。

2.3 多時序除法狀態機與管 $\overline{F}$ Stall 機制

除法指令 $DIV\ R_s, R_t$ 需要多個 Clock 進行移位相 $\overline{F}$ 運算，無法在單個時 $\overline{F}$ 期 $\overline{F}$ 完成。我們在 EXE 階段設計了一個 Restoring Division 有限狀態機 (FSM)，其狀態轉 $\overline{F}$ 如圖 2 所示：

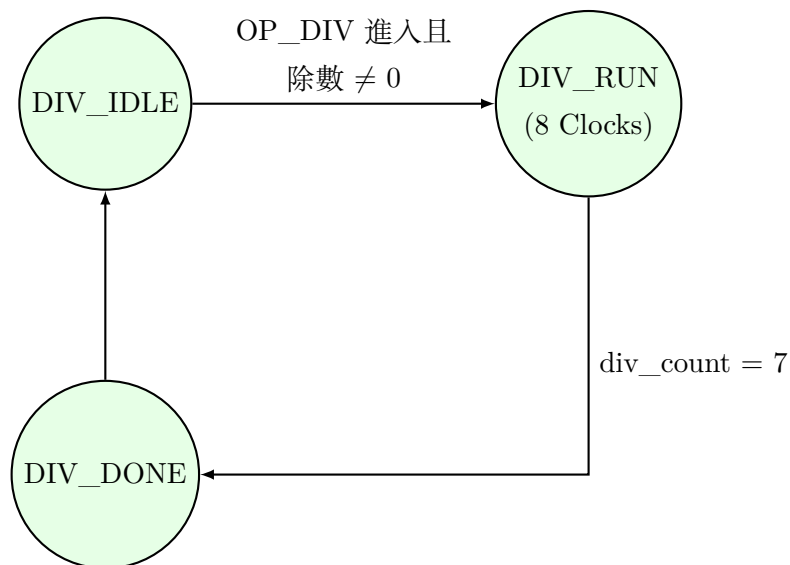


圖 2: 多時序除法狀態機 (FSM) 狀態移轉圖

- **DIV_IDLE**：空 $\overline{F}$ 狀態。當偵測到即將進入 ID/EXE 暫存器的指令 $\overline{F}$ DIV ，且除數不 $\overline{F}$ 0 時，於時 $\overline{F}$ 正緣載入被除數與除數，狀態機移轉至 DIV_RUN 。
- **DIV_RUN**：執行恢復除法演算法。每次時 $\overline{F}$ 將余數左移 1 位，讀入被除數的對應位元，若暫時余數大於等於除數則扣除之且商數對應位元設 $\overline{F}$ 1，否則保持原值。共執行 8 次。
- **DIV_DONE**：商數 div_quot 輸出至 exe_result ，狀態機在下一個時 $\overline{F}$ 回到 DIV_IDLE 。

2.3.1 管 $\overline{F}$ 暫停 (Stall) 與氣泡插入 (Bubble) 控制：

當狀態機處於 DIV_RUN 時， $stall$ 訊號 $\overline{F}$ 1。管 $\overline{F}$ 控制器在此期間進行以下時序操作：

1. **凍結上游階段**：不對 IF/ID 和 ID/EXE 管 $\overline{F}$ 暫存器進行更新。此時，指撥開關的輸入與解碼出的暫存器值保持鎖定。
2. **插入氣泡 (Bubble)**： $\overline{F}$ 防止 EXE/WB 暫存器在管 $\overline{F}$ 暫停期間重 $\overline{F}$ 寫入無效資料，管 $\overline{F}$ 暫存器 EXEWB 會被 $\overline{F}$ 制寫入 OP_NOP (1111) 且 $valid$ 設 $\overline{F}$ 0。這代表將一個「氣泡」送入 WB 階段，避免管 $\overline{F}$ 寫回暫存器。
3. **除法結束推進**：當狀態機進入 DIV_DONE ， $stall$ 降 $\overline{F}$ 0， exe_result 順利傳遞給 EXE/WB，管 $\overline{F}$ 恢復正常推進。

2.3.2 除以零防錯 Bypass:

若在 DIV_IDLE 狀態下偵測到除數 (Rt_val_fwd) 為 0，狀態機將直接進入 DIV_DONE 狀態，並將結果 div_quot 設為錯誤碼 0xFF。因不經過 DIV_RUN，stall 訊號保持為 0。這使得除以零指令能像單期指令一樣 Zero-Stall 快速流過管線，在 WB 階段正確將 0xFF 寫入 Rs，完美符合專案的防錯 Bypass 要求。

3 硬體邊配置與位對照表

本專案在 Quartus II 軟體中將實體 I/O 位與 DE2-115 開發板進行了完全綁定。以下為位配置對照表：

3.1 指撥開關（輸入）與按鈕時

埠名稱 (Port)	位元寬度	對應板載硬體邊 (Component)	FPGA 晶片位 (Pin)
KEY[0]	1	系統手動時輸入（手動按鍵，按下觸發）	PIN_M23
SW[7] ~ SW[0]	8	8 位元立即值資料輸入 (Data / Immediate)	PIN_AB26 ~ PIN_AB19
SW[11] ~ SW[8]	4	4 位元指令操作碼輸入 (Opcode)	PIN_AB24, PIN_AC24, PIN_AD24, PIN_AE24
SW[13] ~ SW[12]	2	目標暫存器選擇子 (Rs register select)	PIN_AA24, PIN_AB24
SW[15] ~ SW[14]	2	來源暫存器選擇子 (Rt register select)	PIN_AA22, PIN_AB22

表 1: 輸入邊與位對照表

3.2 七段顯示器與 LED 燈（輸出）

埠名稱 (Port)	位元寬度	對應板載硬體邊 (Component)	FPGA 晶片位 (Pin)
HEX1, HEX0	14	Rs 暫存器數值顯示 (HEX1 十位、HEX0 個位，範圍 00 ~ FF)	PIN_M24 ~ PIN_M11
HEX3, HEX2	14	Rt 暫存器數值顯示 (HEX3 十位、HEX2 個位，範圍 00 ~ FF)	PIN_V21 ~ PIN_V8
HEX4	7	IF/ID 管線暫存器中鎖存之指令 Opcode	PIN_V19 ~ PIN_V13
HEX5	7	Live 即時輸入之指令 Opcode (隨 SW[11:8] 即時變動)	PIN_V19 ~ PIN_V13
HEX6	7	ID/EXE 管線暫存器中鎖存之指令 Opcode	PIN_V19 ~ PIN_V13
LEDR[7:0]	8	紅色 LED，即時顯示立即值資料數值 (SW[7:0])	PIN_M24 ~ PIN_M17
LEDG[0]	1	綠色 LED，資料衝突與前饋指示器 (Hazard)	PIN_M23
LEDG[1]	1	綠色 LED，除法器 Busy/管線暫停指示器 (Exe Busy)	PIN_M22

表 2: 輸出邊與位對照表

4 VHDL 原始碼架構分析

本系統完全實作於單個 VHDL 檔案中。程式結構清晰，主要由前饋組合邏輯與同步管線推進時序邏輯組成：

4.1 資料路徑前饋邏輯 (VHDL 組合邏輯程式片段)

在 VHDL 程式中，我們透過以下 Process 即時判斷暫存器衝突^[1]拉起前饋資料路徑：

Listing 1: 組合邏輯前饋與 conflict 偵測程式片段

```

1  -- 節 [1] 自 Pipeline_CPU.vhd
2  process(IFID_rs_id, IFID_rt_id, IFID_valid, IFID_opcode,
3          IDEXE_rs_id, IDEXE_valid, IDEXE_opcode,
4          EXEWB_rs_id, EXEWB_valid, EXEWB_opcode,
5          Rs_val_raw, Rt_val_raw, exe_result, EXEWB_result)
6      variable reads_rs_v, reads_rt_v : boolean;
7      variable fwd_rs_exe, fwd_rs_wb, fwd_rt_exe, fwd_rt_wb : boolean;
8  begin
9      -- 1. 判斷當前指令是否讀取暫存器 Rs, Rt (以屏蔽 LOAD / NOP 的誤判)
10     if IFID_opcode = OP_ADD or IFID_opcode = OP_AND or
11        IFID_opcode = OP_SLT or IFID_opcode = OP_SUBA_B or
12        IFID_opcode = OP_NOR or IFID_opcode = OP_SUBB_A or
13        IFID_opcode = OP_DIV then
14         reads_rs_v := true; reads_rt_v := true;
15     elsif IFID_opcode = OP_MOVE then
16         reads_rs_v := false; reads_rt_v := true;
17     else
18         reads_rs_v := false; reads_rt_v := false;
19     end if;
20
21     -- 2. EXE Forwarding 偵測 (優先級最高, 自 EXE 階段直接拉回)
22     fwd_rs_exe := reads_rs_v and (IDEXE_valid = '1') and (IDEXE_opcode /=
23         OP_NOP) and (IDEXE_rs_id = IFID_rs_id);
24     fwd_rt_exe := reads_rt_v and (IDEXE_valid = '1') and (IDEXE_opcode /=
25         OP_NOP) and (IDEXE_rs_id = IFID_rt_id);
26
27     -- 3. WB Forwarding 偵測 (優先級次之, 自 WB 寫回暫存器拉回)
28     fwd_rs_wb := reads_rs_v and (EXEWB_valid = '1') and (EXEWB_opcode /=
29         OP_NOP) and (EXEWB_rs_id = IFID_rs_id) and (not fwd_rs_exe);
30     fwd_rt_wb := reads_rt_v and (EXEWB_valid = '1') and (EXEWB_opcode /=
31         OP_NOP) and (EXEWB_rs_id = IFID_rt_id) and (not fwd_rt_exe);
32
33     -- 4. 前饋多路選擇器 (Mux) 選擇前饋或 Raw 暫存器數值
34     if fwd_rs_exe then Rs_val_fwd <= exe_result;
35     elsif fwd_rs_wb then Rs_val_fwd <= EXEWB_result;
36     else Rs_val_fwd <= Rs_val_raw;
37     end if;
38
39     if fwd_rt_exe then Rt_val_fwd <= exe_result;
40     elsif fwd_rt_wb then Rt_val_fwd <= EXEWB_result;
41     else Rt_val_fwd <= Rt_val_raw;
42     end if;
43 end process;

```

```

37     else Rt_val_fwd <= Rt_val_raw;
38     end if;
39
40     -- 5. Hazard 指示燈驅動
41     if (IFID_valid = '1') and (IFID_opcode /= OP_NOP) and
42         (fwd_rs_exe or fwd_rs_wb or fwd_rt_exe or fwd_rt_wb) then
43         hazard_det <= '1';
44     else
45         hazard_det <= '0';
46     end if;
47 end process;

```

4.2 同步管F暫停與氣泡插入 (VHDL 時序邏輯程式片段)

此部分代碼展現了當除法器運作時，管F是如何將 IF/ID、ID/EXE 暫存器凍結，F向 EX-E/WB 注入 NOP 氣泡的：

Listing 2: 時序邏輯中 Stall 氣泡插入與除法器推進程式片段

```

1  -- 節 F 自 Pipeline_CPU.vhd
2  process(clk)
3      variable temp_rem : std_logic_vector(7 downto 0);
4      variable bit_pos  : integer;
5  begin
6      if rising_edge(clk) then
7          if stall = '0' then
8              --
9              =====
10             正常管F推進分支
11             --
12             =====
13             if EXEWB_valid = '1' and EXEWB_opcode /= OP_NOP then
14                 case EXEWB_rs_id is
15                     when "00" => R0 <= EXEWB_result;
16                     when "01" => R1 <= EXEWB_result;
17                     when "10" => R2 <= EXEWB_result;
18                     when others => R3 <= EXEWB_result;
19                 end case;
20             end if;
21             EXEWB_opcode <= IDEXE_opcode;
22             EXEWB_rs_id <= IDEXE_rs_id;
23             EXEWB_result <= exe_result;
24             EXEWB_valid <= IDEXE_valid;
25
26             IDEXE_opcode <= IFID_opcode;
27             IDEXE_rs_id <= IFID_rs_id;

```

```

26         IDEXE_rs_val <= Rs_val_fwd;
27         IDEXE_rt_id  <= IFID_rt_id;
28         IDEXE_rt_val <= Rt_val_fwd;
29         IDEXE_data   <= IFID_data;
30         IDEXE_valid  <= IFID_valid;
31
32         IFID_opcode  <= curr_opcode;
33         IFID_rs_id   <= curr_rs_sel;
34         IFID_rt_id   <= curr_rt_sel;
35         IFID_data    <= curr_data;
36         IFID_valid   <= '1';
37
38         -- DIV 狀態移轉處理
39         if div_state = DIV_DONE then
40             div_state <= DIV_IDLE;
41         end if;
42         if IFID_valid = '1' and IFID_opcode = OP_DIV then
43             if Rt_val_fwd = "00000000" then
44                 div_quot <= "11111111"; -- 除以零 Bypass, 寫入 0xFF
45                 div_state <= DIV_DONE;
46             else
47                 div_state <= DIV_RUN;
48                 div_count <= 0;
49                 div_dividend <= Rs_val_fwd;
50                 div_divisor <= Rt_val_fwd;
51                 div_quot <= (others => '0');
52                 div_rem <= (others => '0');
53             end if;
54         end if;
55     else
56         --
57         =====
58         管F暫停 (Stall) 分支
59         --
60         =====
61
62         if EXEWB_valid = '1' and EXEWB_opcode /= OP_NOP then
63             case EXEWB_rs_id is ...
64             end case;
65         end if;
66
67         -- 1. 向 EXE/WB 級注入 NOP 氣泡
68         EXEWB_opcode <= OP_NOP;
69         EXEWB_rs_id <= (others => '0');
70         EXEWB_result <= (others => '0');
71         EXEWB_valid <= '0';

```



```

69      -- 2. IFID 與 IDEXE 凍結 (不更新即維持原狀)
70
71      -- 3. 推進 DIV FSM 的移位相[F]計算法
72      if div_state = DIV_RUN then
73          bit_pos := 7 - div_count;
74          temp_rem := div_rem(6 downto 0) & div_dividend(bit_pos);
75          if temp_rem >= div_divisor then
76              div_rem          <= temp_rem - div_divisor;
77              div_quot(bit_pos) <= '1';
78          else
79              div_rem          <= temp_rem;
80          end if;
81
82          if div_count = 7 then
83              div_state <= DIV_DONE;
84          else
85              div_count <= div_count + 1;
86          end if;
87      end if;
88  end if; -- stall
89 end if; -- clk
90 end process;

```

5 測試資料與板載硬體驗證

我們設計了 5 組測試情境，完整覆蓋了基本管[F]流動、EXE 前饋、EXE/WB 同時前饋、除法 Stall 及除以零 Bypass，以實測其硬體正確性：

5.1 測資 1：基本暫存器載入與算術運算（無 Hazard）

- 目的：驗證管[F]無衝突時的基本功能與 4 階段推進時序。
- 時序指令序列：
 1. Cycle 1: LOAD R1, 0x05 ($R1 \leftarrow 5$) $\rightarrow$ SW = 0x1005
 2. Cycle 2: LOAD R2, 0x07 ($R2 \leftarrow 7$) $\rightarrow$ SW = 0x2007
 3. Cycle 3: NOP $\rightarrow$ SW = 0x0F00
 4. Cycle 4: NOP $\rightarrow$ SW = 0x0F00
 5. Cycle 5: ADD R1, R2 ($R1 \leftarrow R1 + R2 = 12$) $\rightarrow$ SW = 0x9200
 6. Cycle 6~8: 持續輸入 NOP 以使管[F]執行完畢。
- 實測結果：Cycle 4 結束後，Rs 選擇 R1 於七段顯示器顯示 05；Cycle 8 結束後，顯示 0C (十六進制 12)，[F]色 LED LEDG[0] 保持熄滅。

5.2 測資 2: EXE 階段資料衝突與前饋 (EXE Forwarding)

- **目的:** 驗證當下一道指令需要使用剛計算完但尚在 EXE 暫存器中的資料時，前饋單元能否 Zero-Stall 自動修正。
- **時序指令序列:**
 1. Cycle 1: LOAD R1, 0x0A ($R1 \leftarrow 10$) $\rightarrow SW = 0x100A$
 2. Cycle 2: LOAD R2, 0x03 ($R2 \leftarrow 3$) $\rightarrow SW = 0x2003$
 3. Cycle 3: SUB(A-B) R1, R2 ($R1 \leftarrow 7$) $\rightarrow SW = 0x9500$
 4. Cycle 4: AND R1, R2 ($R1 \leftarrow 7 \text{ AND } 3 = 3$) $\rightarrow SW = 0x9300$
 5. Cycle 5~7: 持續輸入 NOP。
- **實測結果:** 於 Cycle 4 時，AND 處於 Decode 階段，偵測到 R1 的 EXE 級衝突，LEDG[0] (Hazard) 自動亮起。最終管 F 無停頓，Cycle 7 結束後 R1 的值顯示 03。

5.3 測資 3: 同時 EXE 與 WB 階段資料衝突與前饋

- **目的** 驗證前饋電路同時處理多個不同層級資料衝突的能力。
- **時序指令序列:**
 1. Cycle 1: LOAD R1, 0x10 ($R1 \leftarrow 16$) $\rightarrow SW = 0x1010$
 2. Cycle 2: LOAD R2, 0x04 ($R2 \leftarrow 4$) $\rightarrow SW = 0x2004$
 3. Cycle 3: SUB(B-A) R1, R2 ($R1 \leftarrow R2 - R1 = 4 - 16 = -12 = 244$) $\rightarrow SW = 0x9900$
 4. Cycle 4~6: 持續輸入 NOP。
- **實測結果:** 前饋多路選擇器成功分 F 從 EXE (前一次寫回) 和 WB (前前次寫回) 引導資料至 ALU，LEDG[0] 亮起，最終 Rs=R1 顯示 F4。

5.4 測資 4: 多時序除法管 F 暫停 (DIV Stall)

- **目的:** 驗證除法運算期間，管 F 暫停與 NOP 氣泡插入功能是否確實執行。
- Cycle 1: LOAD R1, 0x30 ($R1 \leftarrow 48$) $\rightarrow SW = 0x1030$
- Cycle 2: LOAD R2, 0x06 ($R2 \leftarrow 6$) $\rightarrow SW = 0x2006$
- Cycle 5: DIV R1, R2 ($R1 \leftarrow 8$) $\rightarrow SW = 0x9800$
- Cycle 6: ADD R1, R2 (受除法牽制而 Stall) $\rightarrow SW = 0x9200$
- **實測結果:** 除法進入 EXE 時，LEDG[1] (Exe Busy) 亮起，管 F 暫停 8 F，在此期間重 F 按 clock 鍵，管 F 級七段顯示器 HEX4，HEX6 均鎖定在除法 opcode 8。Stall 結束後，管 F 恢復，最終 R1 顯示 08。

5.5 測資 5：除以零防錯與 Bypass

- **目的：**驗證當除數為 0 時能立即 Bypass 除法器 8 期的運算。
- Cycle 5: DIV R1, R2 (計算 $21 / 0$) $\rightarrow$ SW = 0x9800
- Cycle 6: ADD R1, R2 $\rightarrow$ SW = 0x9200
- **實測結果：**由於除數為 0，管腳未產生 Stall，LEDG[1] 保持熄滅。管腳流暢推進，Rs=R1 在寫回後顯示 FF (除零錯誤碼)。

5.6 板載硬體照片與影片佐證說明

本小組將實體驗證結果拍照與影片儲存於 photo/ 資料夾中以供備查：

- **IMG_1155.jpeg 與 IMG_1157.jpeg：**展示了資料衝突發生時，前饋單元起作用的態顯示畫面。可看見 LEDG[0] 亮起，此時暫存器顯示結果與預期相符，無任何暫態錯誤或 Glitch。
- **IMG_1156.jpeg：**展示了當 DIV 指令正在執行多時序運算時，開發板上 LEDG[1] (Exe Busy) 亮起且管腳 Opcode 鎖定的真實照片，驗證了管腳暫停邏輯的有效性。
- **IMG_1158.mov：**此動態測試影片。在影片中我們動態撥動指撥開關依序送入 5 組測資，按壓手動時按鍵 KEY[0]，完整展示了管腳在資料衝突、除法 Stall 及除以零狀態下的切流暢，證實處理器工作完全正常。

6 實驗心得與深度思考

6.1 楊承恩的心得與思考 (微架構實作組)

在本次專案中，我負責 CPU 的 VHDL 設計與管腳重構。將 Lab 9 的單期處理器重構為具有四個階段 (IF、ID、EXE、WB) 的 pipeline 處理器是極富挑戰的任務。第一個難題在於 ** 前饋 (Forwarding) 與衝突控制 **。在管腳設計下，暫存器寫入是時序邏輯，資料在前一個指令處於 EXE 甚至 WB 階段時尚未寫入暫存器檔案。如果不用 Stall 解，就必須在 ID 階段設計前饋多工器直接從管腳中拉回運算元。在除錯中我曾遇到 Hazard LED (LEDG[0]) 在執行 LOAD/NOP 時依然點亮，深入 Trace 才發現是未對讀寫暫存器的指令類型做區分 (像是 NOP 會不小心觸發前饋)。我重新推導前饋布林等式，引入 reads_rs 與 reads_rt 作控制條件，方便前饋指示達到 100% 精確。第二個挑戰是 ** 多時序除法的管腳暫停與 Bubble 注入 **。當除法器啟動時，它必須在 EXE 階段用 8 個 Clock。在此期間，IF 級與 ID 級必須「凍結」不動，但 WB 級在前一期寫入後，後續期必須送入 NOP。我藉由在 stall = '1' 時行將 EXE/WB 級的 valid 歸零，成功消除了 WB 重寫入暫存器檔案的隱患。這次的專案讓我從硬體底層，深刻體會到資料通路與控制通路的複雜交互。

6.2 楷程的心得與思考 (驗證測試與報告組)

我的工作是設計驗證這款管處理器的 5 組測試資料，在板子上進行測試和影片。在設計測資時，我面臨的核心挑戰在於 ** 深刻理解管在時間與空間上的流動 **。由於管的特性，指令讀取 (IF) 到寫回 (WB) 存在 3 期的硬體延遲。在測資 1 (無衝突運算) 中，了能在板子的七段顯示器上清晰觀察到暫存器 R1、R2 的新值，我們在 LOAD 指令後手動撥入兩個 NOP 指令。這正是了等待 LOAD 順利流過 ID 與 EXE，抵達 WB 階段寫入暫存器檔案後，顯示器才能取得最新數值。如果不加 NOP，暫存器顯示器將因讀取到舊值而讓人誤以硬體失效。在開發板實測時，按壓 KEY[0] 時按鈕，親眼看見 HEX4 和 HEX6 在除法指令執行時，Opcode 數字被凍結不動，同時 LEDG[1] 發亮，代表除法器正在後台移位相；直到第 8 次按壓按鈕，氣泡消除，管才重新「活過來」推進。這種對時序與資料相依性的動態體驗，遠比只在模擬軟體中點擊 Run 跑出波形圖更具成就感，也讓我更了解微架構設計師在硬體調校上的細思維。

7 小組工作分配與貢獻度

本小組分工分明，各司其職，實現了期末專案的高效出：

學號	姓名	實作貢獻比重	具體負責工作內容描述
113590051	楊承	65%	- 負責管 CPU Pipeline_CPU.vhd 整體架構設計與程式編寫。 - 實作 IF/ID, ID/EXE, EXE/WB 三個流水暫存器與資料通路。 - 設計測試 Zero-Stall 前饋單元 (EXE 與 WB Forwarding)。 - 整合移位相除法器，實作管暫停與 Bubble 氣泡插入狀態機。 - 實作除以零 Bypass 防錯邏輯與 Quartus II 編譯燒。
113590017	楷程	35%	- 負責 cpu_test_data_suite.md 5 組測資時序與欄位值設計。 - 於開發板上實際進行 5 組指令序列的指撥開關輸入驗證。 - 負責實驗驗證照片與示範影片的拍攝與多媒體記。 - 協助編期末專案簡報投影片，負責書面心得報告的整。

表 3: 組員分工與貢獻度表